BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TPHCM

----ꕥ🕮ꕥ----



HCMUTE

COURSE: DATA STRUCTURES AND ALGORITHMS

REPORT

PERFORMANCE COMPARISON OF GCD ALGORITHMS

Member	Student ID
Huỳnh Đức Gia Lạc	25110033
Trương Ngọc Lâm	25110034

Ho Chi Minh City, October 2026

Team Members and Completion

Course: Data Structures and Algorithms

Report title: Performance Comparison of GCD Algorithms

Member	**Student ID**	**Completion**
Huỳnh Đức Gia Lạc	25110033	100%
Trương Ngọc Lâm	25110034	100%

Table of Contents

1. Introduction and Research Objectives 3

2. Working Principles of the Three Algorithms 4

3. Time Complexity Analysis (Big O) 6

4. Experimental Environment and Methodology 8

5. Classification of Input Data 11

6. Execution Time Statistics 14

7. Computational Step Statistics 15

8. Comparison Charts 16

9. Analysis of Performance Differences 21

10. Conclusions and Evaluation 23

1. Introduction and Research Objectives

1.1. Introduction

The Greatest Common Divisor (GCD) is a fundamental concept in mathematics and computer science. It is widely used in number theory, fraction simplification, cryptography, and many computational problems. Finding the GCD of two integers is also a useful problem for studying and comparing the efficiency of different algorithms.

There are several methods for calculating the GCD, including the Euclidean Algorithm, Binary GCD (Stein's Algorithm), and the Extended Euclidean Algorithm. Although all three algorithms produce the same GCD result, they use different computational operations. The Euclidean Algorithm repeatedly uses the modulo operation to reduce the problem, while Binary GCD uses bit shifts, subtraction, and comparisons instead of modulo operations. Extended Euclid follows the same basic process as the traditional Euclidean Algorithm but performs additional calculations to find two coefficients that satisfy Bézout's identity.

These differences can affect both the number of computational steps and the actual execution time. However, an algorithm with fewer steps is not always faster because different operations require different amounts of processing time. The performance of an algorithm can also depend on the input data, hardware environment, and compiler optimizations. Therefore, analyzing theoretical time complexity alone is not enough to determine which algorithm performs best in practice.

In this project, we study and compare the performance of these three GCD algorithms through theoretical analysis and practical experiments. The research is organized into two main stages. In Week 2, we compare the Euclidean Algorithm and Binary GCD based on their execution time and computational steps. In Week 3, we extend the research by studying the Extended Euclidean Algorithm and comparing its performance with the previous algorithms. Through these experiments, we aim to understand how different computational methods affect the efficiency of GCD algorithms under various input conditions.

1.2. Research Objectives

The main objective of this project is to understand how the Euclidean Algorithm, Binary GCD, and Extended Euclid work and how their computational methods affect actual performance. To achieve this goal, we first study the working principles of each algorithm and analyze their theoretical time complexity using Big O notation. This provides a foundation for understanding how the number of operations changes as the input values increase.

In addition to theoretical analysis, we implement the three algorithms in the C programming language and conduct experiments to measure their execution time and computational steps. The experiments use three groups of input data with different mathematical characteristics to observe how each algorithm performs under different conditions. By recording the average,

maximum, and minimum values, we can compare the algorithms more systematically and identify possible differences in their performance.

Another important objective is to investigate the relationship between the number of computational steps and actual execution time. In particular, we want to understand why an algorithm that performs more operations may not always take proportionally longer to execute. Through this research, we hope to identify the advantages and limitations of each algorithm while improving our understanding of algorithm analysis and experimental evaluation.

2. Working Principles of the Three Algorithms

2.1. Euclidean Algorithm

The Euclidean Algorithm is one of the oldest and most commonly used methods for finding the greatest common divisor of two integers. Its main principle is based on the mathematical property that the GCD of two numbers remains unchanged when the first number is replaced by the second number and the second number is replaced by the remainder of their division. This property is expressed by the formula: GCD(a, b) = GCD(b, a % b)

In this formula, the symbol % represents the modulo operation, which returns the remainder after dividing one integer by another. The algorithm repeatedly applies this operation until the second number becomes zero. At that point, the first number contains the greatest common divisor.

For example, when calculating GCD(48, 18), the algorithm first computes 48 % 18 = 12. It then calculates 18 % 12 = 6, followed by 12 % 6 = 0. Since the remainder becomes zero, the algorithm stops and returns 6 as the GCD of 48 and 18.

In our implementation, the Euclidean Algorithm uses a while loop to perform modulo operations and update the values of a and b. Each iteration reduces the problem to a smaller pair of integers, allowing the algorithm to reach the result efficiently. The main advantages of this algorithm are its simplicity, low memory usage, and good performance on many types of input data.

However, the Euclidean Algorithm depends on the modulo operation, which may require more processing resources than basic arithmetic or bitwise operations on some processors. Therefore, although the algorithm usually performs a relatively small number of iterations, its actual execution time can still be affected by the cost of division and the characteristics of the input values.

2.2. Binary GCD (Stein's Algorithm)

Binary GCD, also known as Stein's Algorithm, is another method for calculating the greatest common divisor of two integers. Unlike the traditional Euclidean Algorithm, Binary GCD

mainly uses bit shifts, subtraction, and comparisons instead of the modulo operation. The algorithm is based on several mathematical properties related to even and odd numbers.

When both numbers are even, their GCD can be calculated by dividing both numbers by 2 and multiplying the result by 2. This property can be written as GCD(a, b) = 2 × GCD(a/2, b/2). When one number is even and the other is odd, the even number can be divided by 2 without changing the GCD. When both numbers are odd, the larger number can be reduced by subtracting the smaller number, allowing the algorithm to continue with smaller values.

In our implementation, the algorithm first checks whether either input value is zero. If one value is zero, the other value is returned immediately. Otherwise, the algorithm removes common factors of 2 from both numbers using right bit shifts (>>) and records the number of common factors removed. After that, it continues removing factors of 2, comparing the two values, swapping them when necessary, and subtracting the smaller value from the larger one. This process continues until one of the numbers becomes zero. Finally, the result is restored using a left bit shift (<<) based on the number of common factors of 2 removed earlier.

For example, when calculating GCD(48, 18), both numbers are initially even, so the problem becomes 2 × GCD(24, 9). Since 24 is still even, it can be repeatedly divided by 2 until it becomes 3. The problem is then reduced to GCD(3, 9), followed by GCD(3, 6) and GCD(3, 3). Therefore, the final result is 2 × 3 = 6.

The main advantage of Binary GCD is that bitwise operations can be executed efficiently by many processors. This makes the algorithm particularly interesting when processing numbers with many factors of 2. However, Binary GCD may perform more individual operations than the Euclidean Algorithm because it uses several shifts, comparisons, and subtractions to reduce the values. As a result, the use of simpler operations does not always guarantee a shorter execution time. Its actual efficiency depends on the balance between the number of operations and the cost of performing each operation.

2.3. Extended Euclidean Algorithm

The Extended Euclidean Algorithm is an extension of the traditional Euclidean Algorithm. Besides finding the GCD of two integers, it also calculates two integer coefficients, x and y, that satisfy Bézout's identity: a × x + b × y = GCD(a, b)

The algorithm follows the same basic reduction process as the standard Euclidean Algorithm by repeatedly using the modulo operation. However, it performs additional calculations to determine the coefficients x and y. These coefficients are useful in number theory, especially for finding modular inverses and solving linear Diophantine equations.

In our implementation, Extended Euclid uses recursion. When the second input value becomes zero, the algorithm reaches its base case and returns the first value as the GCD. At the same time, the coefficients are initialized as x = 1 and y = 0. During the recursive return process, the algorithm updates the coefficients using the formulas x = y1 and y = x1 - (a / b) × y1. These

calculations allow the algorithm to determine the coefficients that satisfy Bézout's identity for the original input values.

For example, consider GCD(99, 78). By repeatedly applying the Euclidean Algorithm, we obtain the divisions $99 = 78 \times 1 + 21$, $78 = 21 \times 3 + 15$, $21 = 15 \times 1 + 6$, $15 = 6 \times 2 + 3$, and $6 = 3 \times 2 + 0$. Therefore, the GCD is 3. By working backward through these equations, the Extended Euclidean Algorithm determines that $x = -11$ and $y = 14$, satisfying the equation $99 \times (-11) + 78 \times 14 = 3$.

Although Extended Euclid follows the same sequence of modulo-based reductions as the traditional Euclidean Algorithm, it requires additional arithmetic operations to calculate the coefficients. In our recursive implementation, these calculations also involve recursive function calls and returning through the recursion. Consequently, Extended Euclid may require more execution time even when both algorithms perform the same number of modulo-based reduction steps. This makes it a useful algorithm for studying the difference between the number of computational steps and the actual cost of execution.

3. Time Complexity Analysis (Big O)

3.1. Time Complexity of the Euclidean Algorithm

The Euclidean Algorithm has a time complexity of $O(\log(\min(a, b)))$, where a and b are positive integers. This logarithmic complexity comes from the fact that each modulo operation significantly reduces the size of the problem. As the algorithm continues, the values become smaller until the second number reaches zero.

One of the cases requiring the greatest number of iterations occurs when the two input values are consecutive Fibonacci numbers. In this situation, each modulo operation reduces the values relatively slowly compared with many other input pairs. However, even in such cases, the total number of iterations grows logarithmically with the smaller input value. This makes the Euclidean Algorithm efficient for finding the GCD of relatively large integers.

3.2. Time Complexity of Binary GCD

Binary GCD has a time complexity of $O(\log(\max(a, b)))$ when counting operations on fixed-width machine integers. The algorithm reduces the input values through bit shifts and subtraction. When a number is even, a right bit shift divides it by 2. When both numbers are odd, subtraction reduces the difference between them, allowing the algorithm to continue with smaller values.

Although Binary GCD and the traditional Euclidean Algorithm both have logarithmic time complexity, they use different computational methods. Binary GCD may require several bit shifts, comparisons, or subtractions to achieve a reduction that the Euclidean Algorithm can

perform using a single modulo operation. Therefore, the number of operations performed by Binary GCD may be greater even though both algorithms belong to the same complexity class.

However, bit shifts and subtraction can be less expensive than modulo operations on some processors. This means that a greater number of operations does not necessarily result in proportionally longer execution time. The actual performance depends on the cost of the operations and the characteristics of the input values.

3.3. Time Complexity of Extended Euclid

The Extended Euclidean Algorithm has a time complexity of O(log(min(a, b))), which is the same as the traditional Euclidean Algorithm. Both algorithms follow the same modulo-based reduction process, so the number of these reductions is identical when the same input values are used.

However, Extended Euclid performs additional calculations during each recursive step to determine the coefficients x and y. These calculations include multiplication, subtraction, and coefficient updates. The recursive implementation also introduces additional function-call operations.

Although these operations increase the amount of computation required, they do not change the overall logarithmic time complexity. As a result, Extended Euclid can have a longer execution time than the traditional Euclidean Algorithm even when both algorithms have the same theoretical complexity and the same number of modulo-based reduction steps.

3.5. Overall Comparison of Time Complexity

The following table summarizes the main differences between the three algorithms in terms of time complexity, computational operations, and characteristics.

Algorithm	**Time Complexity**	**Main Operations**	**Characteristics**
Euclidean Algorithm	O(log(min(a,b)))	Modulo	Simple implementation and relatively few reduction steps
Binary GCD	O(log(max(a,b)))	Bit shifts, subtraction, comparisons	Avoids modulo operations but may require more individual steps
Extended Euclid	O(log(min(a,b)))	Modulo, multiplication, subtraction, recursion	Calculates the GCD and Bézout coefficients

As shown in the table, all three algorithms have logarithmic time complexity with respect to input magnitude. However, their computational methods differ significantly. The Euclidean Algorithm mainly relies on modulo operations, Binary GCD uses bitwise operations and subtraction, while Extended Euclid performs additional coefficient calculations.

These differences explain why algorithms with similar Big O complexity can still have different execution times. An algorithm may perform fewer operations but spend more time on each one, while another may perform more operations that are individually less expensive. Therefore, theoretical complexity alone cannot determine which algorithm will be the fastest in practice.

4. Experimental Environment and Methodology

4.1. Experimental Environment

To evaluate the performance of the three algorithms, we implemented the benchmark programs in the C programming language. The programs use standard C libraries, including stdio.h, stdlib.h, and time.h, to handle input data generation, output, and time measurement. Execution time is measured using the clock() function, while computational steps are calculated using separate step-counting functions.

In our experiments, each input group contains 10 pairs of positive integers. To improve the accuracy of time measurement, each algorithm is executed 1,000,000 times for each input pair. The experiments are divided into three groups based on different mathematical characteristics of the input values. For each group, we record the average, maximum, and minimum execution times, as well as the corresponding statistics for computational steps.

The experimental process is organized into two main stages. In Week 2, we evaluate the Euclidean Algorithm and Binary GCD to understand the differences between modulo-based and bitwise approaches. In Week 3, we introduce the Extended Euclidean Algorithm and compare its performance with the previous algorithms. The collected results are organized into Excel spreadsheets and presented through comparison charts to make the differences easier to observe and analyze.

4.2. Execution Time Measurement

GCD algorithms generally require very little time to complete a single calculation. Therefore, measuring the execution time of only one function call may produce inaccurate or unstable results because the measured duration can be close to the timer's resolution. To reduce this problem, our benchmark repeatedly executes each algorithm using the same input pair and calculates the average execution time.

For each experiment, the program first generates a pair of integers a and b according to the conditions of the selected input group. It then records the starting clock value using the clock() function. The selected GCD algorithm is executed 1,000,000 times with the same input pair, after which the ending clock value is recorded. The total measured time is divided by the number of repetitions to obtain the average execution time for one function call.

The execution time is calculated using the following formula:

T = (end - start) / (CLOCKS_PER_SEC × REPEAT)

In this formula, start and end represent the clock values recorded before and after the repeated executions. CLOCKS_PER_SEC is used to convert clock ticks into seconds, while REPEAT represents the number of executions, which is set to 1,000,000 in our benchmark.

After measuring all 10 input pairs in a group, the program calculates the average execution time by adding the measured values and dividing the total by 10. The maximum and minimum execution times are also recorded to show the variation in performance across different input pairs. This process is repeated for each algorithm and each input group, allowing us to compare how execution time changes under different conditions.

4.3. Computational Step Measurement

In addition to execution time, our experiments also measure the number of computational steps performed by each algorithm. This helps us understand how different algorithms reduce the input values and whether the number of steps is directly related to execution time. To perform this measurement, the benchmark uses three separate functions: gcdSteps() for the Euclidean Algorithm, BinaryGCDSteps() for Binary GCD, and extendedEuclidSteps() for Extended Euclid.

The three functions use different rules for counting computational steps, as summarized in the following table.

Algorithm	**Step-Counting Function**	**What Is Counted**
Euclidean Algorithm	gcdSteps()	One step for each modulo-based reduction
Binary GCD	BinaryGCDSteps()	Steps for bit shifts, swaps, and subtractions
Extended Euclid	extendedEuclidSteps()	One step for each modulo-based reduction, excluding coefficient updates

For the Euclidean Algorithm, a computational step is counted whenever one modulo-based reduction is completed. The function repeatedly calculates the remainder and updates the input values until the second number becomes zero. The total number of iterations is then returned as the number of steps.

For Binary GCD, the step-counting function follows a different approach because the algorithm does not use modulo operations in its main calculation. Instead, the function increases the step counter when performing certain operations, including right bit shifts, swaps, and subtractions.

As a result, Binary GCD may have a larger number of recorded steps than the Euclidean Algorithm, even when both algorithms calculate the GCD of the same input pair.

For Extended Euclid, the step-counting function counts modulo-based reductions in the same way as the traditional Euclidean Algorithm. Therefore, when both algorithms are tested with identical input values, they produce the same recorded number of steps. However, Extended Euclid also performs additional calculations to update the coefficients x and y, and these operations are not included in the recorded step count.

This distinction is important when interpreting the experimental results. The number of recorded steps does not represent the same amount of computational work for all three algorithms. A Binary GCD step may involve a simple bit shift or subtraction, while a Euclidean step involves a modulo operation. Similarly, Extended Euclid performs additional calculations that are not reflected in its step counter. Therefore, step count should be analyzed together with execution time rather than being used as the only measure of efficiency.

For each group of 10 input pairs, the benchmark calculates the average, maximum, and minimum numbers of steps. The step-counting functions are executed separately from the timed loops, so their additional calculations are not directly included in the measured execution time. By comparing the recorded steps with the execution times, we can better understand why algorithms with different numbers of steps may have similar performance or why algorithms with the same number of steps may require different amounts of time.

4.4. Performance Evaluation and Experimental Limitations

The performance evaluation focuses on two main measurements: execution time and computational steps. These measurements are collected for each algorithm and organized according to the three input groups. By comparing the average, maximum, and minimum values, we can observe differences in performance and examine how the mathematical properties of the input data affect each algorithm.

However, several factors may influence the experimental results. First, the benchmark programs use randomly generated input data, so the algorithms may not always be tested using exactly the same number pairs. Although they are evaluated using the same input categories, differences between individual input values can still affect the number of iterations and execution time. For a more controlled comparison, the same predefined dataset should be used for all three algorithms.

Another limitation is that execution time may be affected by the hardware environment, compiler optimization settings, timer accuracy, and other processes running on the computer. Because the measured execution times are very small, even minor changes in the execution environment may influence the results. In addition, the benchmark must ensure that computed results remain observable so that compiler optimizations do not remove the calculations being measured.

Finally, computational steps are counted differently across the algorithms. The Euclidean Algorithm and Extended Euclid count modulo-based reductions, while Binary GCD counts several types of binary operations. Therefore, comparing their step counts directly without considering the cost of each operation may lead to incorrect conclusions.

For these reasons, the experimental results should be evaluated together with theoretical analysis. The purpose of the experiments is not only to identify which algorithm executes faster in a particular test, but also to understand the reasons behind the differences in performance.

5. Classification of Input Data

5.1. Group 1: Random Numbers

The first input group consists of randomly generated integer pairs. In the benchmark, the values of a and b are generated using the expressions rand() % 90000 + 10001, which are intended to produce integers ranging from 10,001 to 100,000. The two numbers are generated independently, without requiring any specific relationship between them.

Because there are no restrictions on the GCD, this group may contain relatively prime numbers, numbers with common factors greater than 1, and pairs with different numerical differences. These characteristics make the group useful for evaluating the general performance of the algorithms under ordinary input conditions.

The purpose of this group is to observe how the three algorithms behave when the input values do not follow a special mathematical pattern. By comparing their execution times and computational steps, we can obtain an overall view of their performance across different randomly generated number pairs.

5.2. Group 2: Coprime Numbers

The second input group consists of pairs of relatively prime integers. Two numbers are considered coprime when their greatest common divisor is equal to 1, meaning that they do not share any positive factor other than 1.

In our benchmark, the program randomly generates two integers within the selected range and checks their GCD. If the result is not equal to 1, the program generates another pair until the condition is satisfied. This ensures that every input pair in the group consists of two relatively prime numbers.

For example, 17 and 31 are relatively prime because their only common positive divisor is 1. However, the actual benchmark uses larger values generated within its specified range.

Coprime numbers are useful for studying the behavior of GCD algorithms because they continue reducing the input values until the GCD of 1 is reached. Some coprime pairs may require many iterations, especially when their values are related to consecutive Fibonacci

numbers. However, not every coprime pair requires a large number of steps, since the exact number of iterations depends on the relationship between the two values.

The purpose of this group is to compare how efficiently the algorithms process numbers with a GCD of 1. The results allow us to examine differences in the number of reductions and the computational cost of each algorithm under this condition.

5.3. Group 3: Powers-of-Two-Related Numbers

The third input group consists of integer pairs generated from a common base multiplied by powers of 2. This group is designed to study how the algorithms handle numbers with common factors and simple relationships involving powers of 2.

In the benchmark, the program first generates a common base value using rand() % 10000 + 10001, which is intended to produce a number between 10,001 and 20,000. It then generates two exponents, x and y, using rand() % 3, so each exponent can take the value 0, 1, or 2. The two input numbers are calculated using the formulas a = common × 2^x and b = common × 2^y.

For example, if the common base value is 12,000, with x = 1 and y = 2, the generated numbers are a = 24,000 and b = 48,000. In this case, the GCD is 24,000. This example shows that both values share a common factor and their ratio is related to a power of 2.

This group is particularly useful for evaluating Binary GCD because the algorithm removes factors of 2 through bit-shifting operations. Since bit shifts can be performed efficiently on many processors, Binary GCD may benefit from this type of input. However, the traditional Euclidean Algorithm can also process these pairs efficiently because one number is a simple multiple of the other, allowing the GCD to be found with very few modulo operations.

Therefore, the purpose of this group is to compare the performance of bitwise and modulo-based approaches when the input values have simple relationships involving powers of 2. The results help determine whether Binary GCD gains a practical performance advantage in these cases or whether the Euclidean Algorithm remains competitive because of its small number of iterations.

5.4. Summary of Input Data Groups

The following table summarizes the characteristics and research purposes of the three input groups used in the experiments.

Input Group	Characteristics	Research Purpose
Random Numbers	Random integer pairs, intended range 10,001–100,000	Evaluate general algorithm performance
Coprime Numbers	Integer pairs with GCD(a,b) = 1	Evaluate performance when the numbers have no common factor greater than 1
Powers-of-Two-Related Numbers	Integer pairs generated from a common factor multiplied by powers of 2	Study the influence of factors of 2 and bit-shifting operations

Each input group contains 10 pairs of positive integers with different mathematical characteristics. The Random Numbers group represents general cases without specific restrictions on the GCD. The Coprime Numbers group focuses on pairs whose GCD is equal to 1, while the Powers-of-Two-Related Numbers group examines pairs that share a common factor and have a relationship involving powers of 2.

Using these three groups provides a broader evaluation of algorithm performance than testing only one type of input. By comparing the execution time and computational steps across different groups, we can observe how input characteristics influence the efficiency of each algorithm.

Overall, the combination of input classification, execution time measurement, and computational step analysis helps us evaluate the advantages and limitations of the Euclidean Algorithm, Binary GCD, and Extended Euclid. It also supports the main objective of the project, which is to understand the relationship between theoretical time complexity, the number of computational steps, and actual execution performance.

6. Execution Time Statistics

Sections 6.1 to 6.3 report the workbook run. The separate terminal run is shown in Figures 3 to 5 in Section 8. Each group contains 10 recorded pairs. For one pair, the measured duration of 1,000,000 repeated calls is divided by 1,000,000. The group average is the mean of those 10 values. Maximum and minimum are the largest and smallest per-pair average times, rather than the fastest or slowest individual call.

Times are reported in nanoseconds per call for readability. One nanosecond (ns) equals 0.000000001 seconds. The workbook stores the same results in seconds. All values below come from its algorithm time sheets and Times Comparison sheet.

6.1. Random Pairs

Algorithm	Average (ns)	Maximum (ns)	Minimum (ns)
Euclidean Algorithm	20.4	31	13
Binary GCD	46.2	53	36
Extended Euclid	47.4	59	24

6.2. Coprime Pairs

Algorithm	Average (ns)	Maximum (ns)	Minimum (ns)
Euclidean Algorithm	20.6	35	12
Binary GCD	46.7	53	38
Extended Euclid	45.1	67	24

6.3. Powers of Two Times a Common Factor

Algorithm	Average (ns)	Maximum (ns)	Minimum (ns)
Euclidean Algorithm	2.5	3	1
Binary GCD	2.3	4	1
Extended Euclid	4.5	7	3

Euclid has the lowest average time for random and coprime pairs. For the third group, Binary GCD has the lowest recorded average, 2.3 ns, compared with 2.5 ns for Euclid. The difference is only 0.2 ns per call. Without repeated independent runs, this small difference does not establish a reliable general advantage.

7. Computational Step Statistics

Figure 2 in Section 8 presents the workbook step statistics for the three input groups. Figures 3 to 5 show the separate terminal run. Each step value describes one GCD calculation for one pair and is not multiplied by the timing repetition count. The average, maximum, and minimum values are calculated from the 10 pairs in each group.

7.1. Meaning of the Recorded Steps

gcdSteps() and extendedEuclidSteps() each count one modulo-based reduction per loop iteration. For identical inputs they therefore return the same count. Their different averages here result from different recorded input pairs. Extended Euclid is not using fewer reductions than Euclid on the same pair.

BinaryGCDSteps() counts one step when both even inputs are shifted together, one for each later right shift, one for each swap, and one for each subtraction. It does not count every comparison or loop test. extendedEuclidSteps() excludes coefficient updates, recursive calls, and recursive returns. These counters measure different operations, so their totals are not directly comparable as CPU instruction counts.

8. Comparison Charts

Figures 1 and 2 reproduce the supplied workbook comparison charts, including average, maximum, and minimum values. Figures 3 to 5 show the separate terminal run for the three algorithms. The chart time axis and terminal output use seconds. Sections 6.1 to 6.3 express the workbook times in nanoseconds, where 1 ns = 0.000000001 seconds. The workbook and terminal runs are compared separately because their inputs differ.

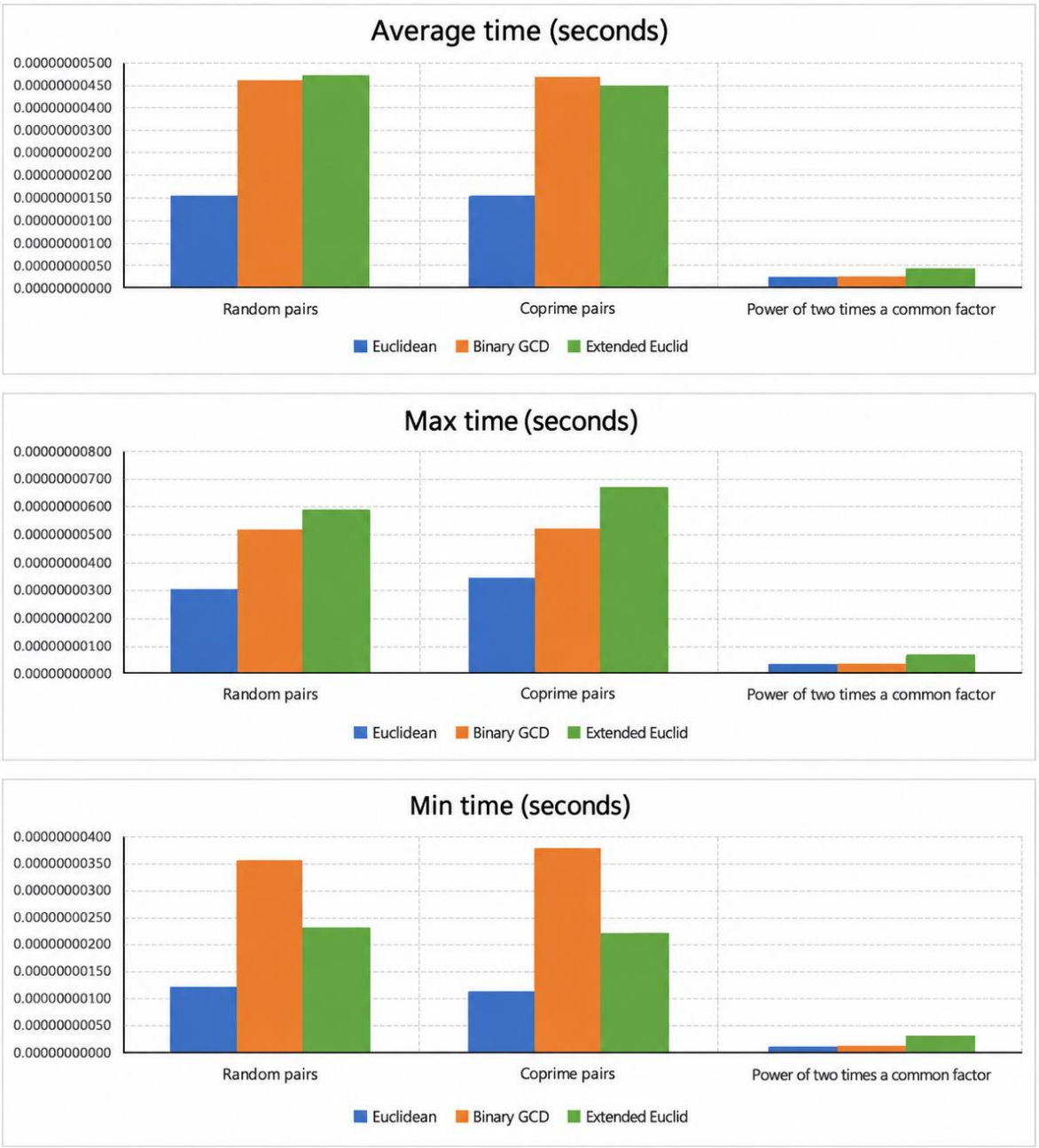


Figure 1. Execution time comparison for the three GCD algorithms.

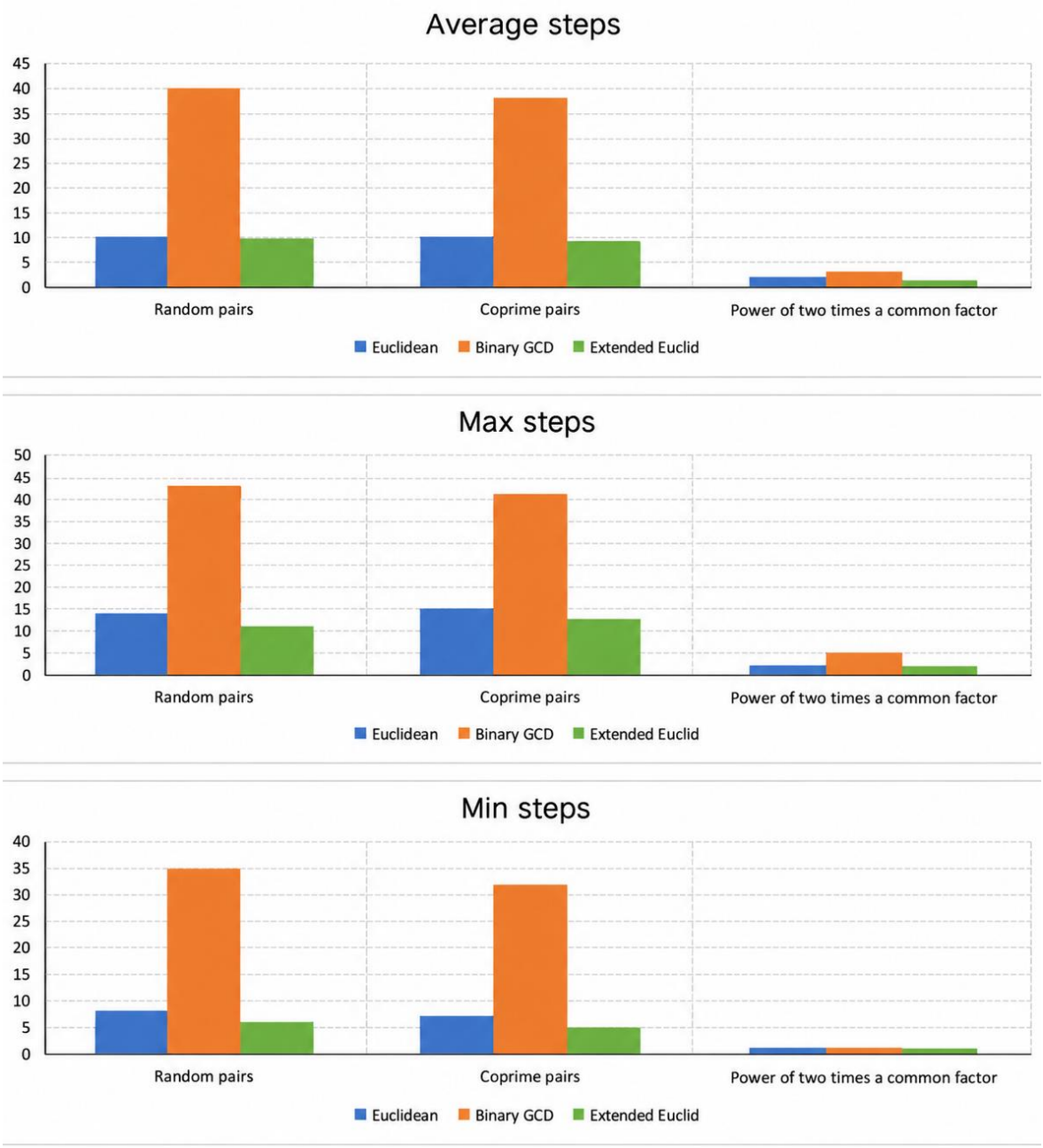


Figure 2. Step comparison for the three GCD algorithms.

The third input group requires much less time and fewer recorded steps than the other groups. Binary GCD records the most steps in all three groups. Extended Euclid records fewer reduction steps than Euclid in these samples, but its average execution time is higher in all three groups. This illustrates why step definitions and the operations performed inside each algorithm must be considered together.

8.1. Euclidean Algorithm Terminal Output

The screenshot shows per-pair time and steps, followed by Average, Max, and Min summaries. Its average times are 18.8 ns, 17.7 ns, and 1.9 ns for the three groups, respectively.

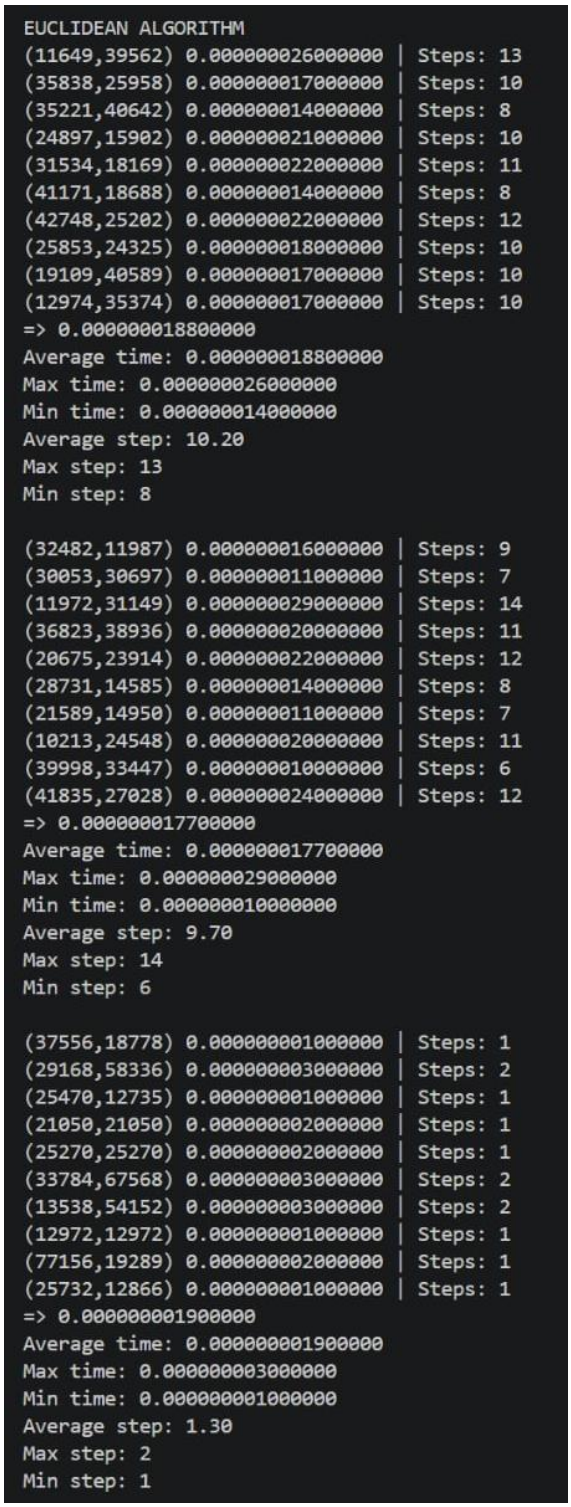


Figure 3. EuclideanTerminal.png showing the three input groups.

8.2. Binary GCD Terminal Output

The three output blocks record average times of 38.3 ns, 43.5 ns, and 3.5 ns. The corresponding average step counts are 35.9, 38.2, and 3.2.

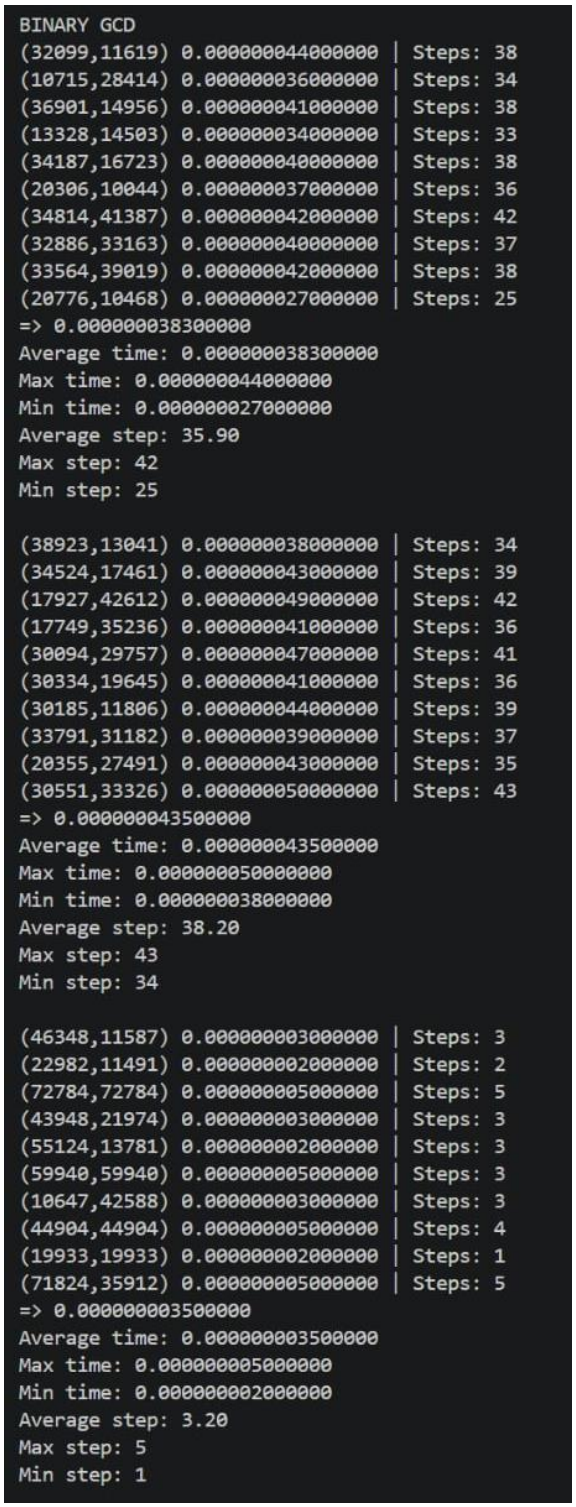


Figure 4. BinaryGCDTerminal.png showing the three input groups.

8.3. Extended Euclid Terminal Output

The average times are 52.2 ns, 55.5 ns, and 4.2 ns. Its average recorded reduction counts are 9.4, 9.8, and 1.4. Coefficient updates and recursive returns are not included in this counter.

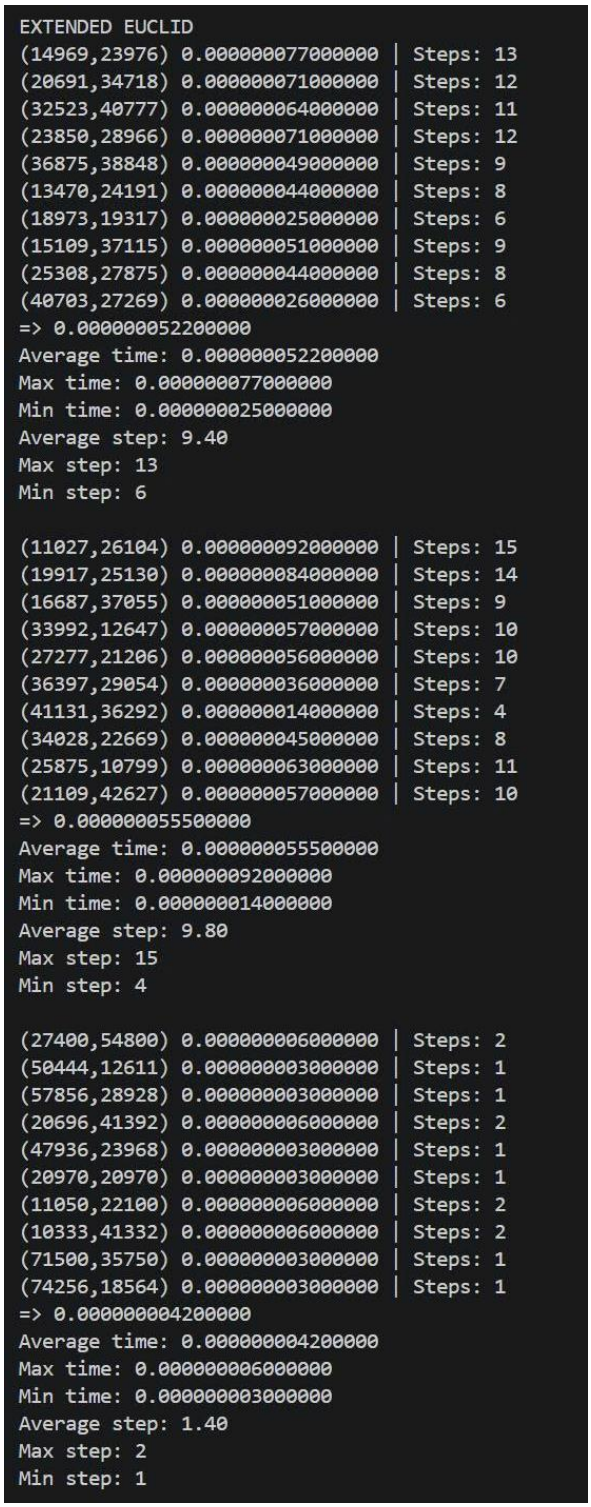


Figure 5. ExtendedEuclidTerminal.png showing the three input groups.

9. Analysis of Performance Differences

9.1. Euclid and Binary GCD

For random pairs, Binary GCD records about 3.82 times as many steps as Euclid (40.1 / 10.5), while its average time is about 2.26 times as high (46.2 / 20.4). For coprime pairs, the corresponding ratios are about 3.62 for steps and 2.27 for time. The time ratio is smaller than the step ratio, which is consistent with the algorithms doing different work per counted step.

The supplied code explains the extra operations in Binary GCD. gcd() performs one remainder calculation per reduction. BinaryGCD() can perform several shifts, tests, a swap, and a subtraction before reaching the next smaller pair. In these samples, avoiding modulo does not make Binary GCD faster for random and coprime pairs.

AnalyzePerformance.pdf discusses the possible benefit of bit shifts and subtraction on hardware with expensive division. That is a conditional explanation, not a measured result for this experiment. The supplied measurements do not establish the cost of individual CPU instructions or the presence of a particular hardware divider, so no exact clock-cycle claim is made here.

9.2. Euclid and Extended Euclid

Extended Euclid has average times about 2.32, 2.19, and 1.80 times those of Euclid for random pairs, coprime pairs, and the third group, respectively. These ratios compare the recorded group means. They do not isolate the cost of the algorithms on identical inputs.

The function extendedEuclid() recursively calls extendedEuclid(b, a % b, &x1, &y1). On returning, it assigns x = y1 and computes y = x1 - (a / b) * y1. This adds coefficient arithmetic and recursive work to the remainder-based reduction. The separate step counter does not include that extra work. This explains how Extended Euclid can take longer while showing a similar or smaller recorded reduction count in the current samples.

9.3. Effect of the Third Input Group

In this group, a = common * 2^x and b = common * 2^y, with x and y each in {0, 1, 2}. One number is a multiple of the other. If a is already a multiple of b, Euclid finishes after one remainder calculation. If the smaller value is supplied first, the first remainder exchanges their roles and a second reduction finishes the calculation. This matches the observed minimum of 1 and maximum of 2 reductions for both Euclidean counters.

Binary GCD also benefits from the simple relationship between the values because shifts remove factors of 2. Its average falls to 2.9 counted steps and 2.3 ns. Euclid remains close at 1.6 reductions and 2.5 ns. The data therefore support a strong effect of input structure on all three algorithms, rather than an exclusive benefit for Binary GCD.

9.4. Limits of the Comparison

The programs generate new pairs separately for each algorithm, using rand() and a time-based seed. The workbook confirms that the recorded pairs differ. Differences between group averages therefore include both algorithm behavior and input differences. The third group is also limited to very small exponents, so it does not test every possible power-of-two case.

Timing uses clock() and CLOCKS_PER_SEC as implemented by the benchmark environment. No separate wall-clock timer, compiler flags, or repeated-run uncertainty measurements are supplied. The very small reported per-call values are obtained by dividing a longer measured loop duration, not by timing a single nanosecond-scale call directly.

In BenchmarkEuclideanAndBinaryGCD.c, result is an ordinary int and is not used after the timed loop. An optimizing compiler may remove work whose result is unused. BenchmarkExtendedEuclid.c stores its result in a volatile unsigned int, but the coefficient values are not printed or otherwise checked in the timed test. Without compiler settings and generated machine code, the measurements cannot prove that every intended operation remained in the timed loops. These limitations restrict the conclusions to the recorded benchmark results.

9.5. Comparison of the Two Recorded Runs

For random pairs, both runs show the same average-time order: Euclid, Binary GCD, then Extended Euclid. For coprime pairs, Euclid is fastest in both runs, but Binary GCD and Extended Euclid exchange places. The workbook gives 45.1 ns for Extended Euclid and 46.7 ns for Binary GCD. The terminal run gives 55.5 ns and 43.5 ns, respectively.

For the third group, Binary GCD narrowly leads in the workbook, while Euclid leads in the terminal run. The input pairs change between algorithms and between runs. These observations show why a single ranking should not be treated as a general rule. The consistent findings are that the third input structure reduces time and steps strongly, Binary GCD has more counted operations, and Extended Euclid takes longer than Euclid in both recorded runs.

10. Conclusions and Evaluation

10.1. Euclidean Algorithm

The Euclidean Algorithm is the strongest practical choice when only the GCD is needed in these recorded tests. Its advantages are a short iterative implementation, few reduction steps, and the lowest average time for random and coprime pairs in both runs. It also has the lowest average time for the third terminal group. Its limitation is its dependence on remainder operations, whose cost depends on the execution environment.

10.2. Binary GCD

Binary GCD avoids modulo in its main calculation and uses shifts, subtraction, and comparisons. This is its main implementation advantage. It has the lowest average time for the third workbook group, but that small advantage is not repeated in the terminal run. Its disadvantages in the recorded data are more counted operations and longer average times than Euclid for random and coprime pairs. Simpler individual operations do not automatically make the complete algorithm faster.

10.3. Extended Euclid

Extended Euclid returns both the GCD and the coefficients x and y in ax + by = GCD(a, b). This additional output is its main advantage and supports the uses described in AnalyzeStandardEuclideanAndExtendedEuclidean.pdf, including modular inverses and linear equations. Its disadvantages are extra arithmetic, recursion, and higher average time than Euclid in all groups of both recorded runs. It is appropriate when the coefficients are needed, while Euclid is simpler when only the GCD is required.

10.4. Overall Conclusion

The workbook and terminal results show that input structure and the work performed per step both matter. All three algorithms are much faster on the common-factor and power-of-two pairs than on the other groups. Euclid is fastest for random and coprime pairs in both runs. The fastest algorithm for the third group changes between runs, so the data do not support a general winner for that category. Extended Euclid provides additional mathematical information at a higher measured cost than Euclid. A controlled follow-up should reuse identical pairs, keep all required outputs observable, record compiler settings, and repeat the complete timing experiment.